

Cache Partitioning for Mitigating Timing Side-Channel Attacks in LLM Serving Systems

Zixuan Pang^{1*}, Wenhao Wang², Yong Liao¹

¹School of Cyber Science and Technology, University of Science and Technology of China, Hefei, China

²Institute of Information Engineering, CAS, Beijing, China

*pangzixuan@mail.ustc.edu.cn

Abstract—The continuous development of Large Language Models (LLMs) has led to a significant demand for improving their inference efficiency. However, recent studies indicate that shared Key-Value (KV) cache, a widely used optimization techniques, can be exploited as a side-channel to recover user's prompts in multi-users settings, which is confirmed to be widespread across various LLM inference services, posing threats to user's privacy. This paper proposes a user-boundary-based cache partition mechanism to mitigate the above prompt leakage induced by shared caches. Our design partitions the KV cache into different prefix trees by user identities, preventing attackers from detecting cache hits and recovering shared prompt prefixes in the caches. The experimental results shows that cache partitioning mechanism can eliminate the timing difference, and the system's overhead remains within an acceptable range (6.44% in Time-To-First-Token and 3.06% in throughput) across different models and scenarios.

Keywords—LLM; Timing side-channel attack; Cache partition

I. INTRODUCTION

With the development of large language models (LLMs), their applications are broadening across various domains, including healthcare, artistic creation, and finance. However, the computational and storage overhead of LLM inference is quite expensive, prompting researchers to propose optimization methods, including quantization [6], model distillation [8], and sharing KV caches [4], [19]. However, the security of these optimization techniques has not yet been clear. In particular, sharing KV caches among multiple users, such as prefix caching, may lead to variations in user request response times due to memory sharing, scheduling, and eviction processes, thereby creating timing side channels that could potentially be exploited to steal other users' prompt information.

Recently researchers identified a timing side-channel leakage [12] of the existing KV prefix cache shared inference technique in multi-user environments. KV cache is a tensor cache generated during LLM inference computations to facilitate repeated calculations, while prefix caching is a mechanism that maps user-input tokens to corresponding KV caches and organizes them in a prefix tree based on the input sequence. When a prompt is input, the inference backend caches its tokens and corresponding KV cache in a prefix tree. If a subsequent prompt shares the same prefix, the cached KV is reused, reducing response time. This creates a timing difference that can be exploited by an attacker. By inputting guessed prefix information into the inference framework and observing whether the response time decreases, the attacker can infer whether other users in the inference framework have entered the same prefix, thereby potentially compromising

private data. This vulnerability has been proven to be widespread in LLM inference optimization techniques [4], [19] and online LLM servings.

To mitigate this security risk, we propose a user-boundary KV cache [7] partition mechanism that ensures that KV cache is isolated for each user, and only requests from the same user are cached together in a prefix tree. So attackers can't infer other users' prompts through response time. This mechanism prevents timing side-channel attacks by ensuring that prefix caches are shared solely among requests from the same user, thereby eliminating cross-user timing side-channel vulnerabilities. Our approach maintains the mapping between tokens and KV caches while isolating prefix matching, ensuring that only requests with the same authentication token share the same prefix. Since prefix caching is commonly used in single-user multi-turn dialogues or document-sharing scenarios, this user isolation mechanism introduces minimal overhead in real-world applications.

In the security evaluation, we compared the impact on individual request response times between the original non-isolated system and the cache partitioned system when cross-user shared prefixes were present on different size models. The results showed that in the partitioned system, whether different users issued prompts with the same prefix had no effect on request response time, effectively eliminating the timing side channel. In the performance evaluation, we used various datasets across different models to assess the performance overhead of the partitioned cache system based on three metrics: TTFT, throughput, and system startup time. The results indicated that in single-user scenarios, the average TTFT overhead was 6.4%, the throughput overhead was 3.06%, and the startup time remained nearly unchanged.

In summary, the contributions of this paper are as follows:

- We propose a user-boundary-based LLM inference cache partitioning system that eliminates timing side-channel vulnerabilities potentially caused by the widely used multi-user shared cache optimization mechanisms in current LLM inference.
- We designed a registration module that provides a unique identifier for each user, isolating each user's requests into separate cache partitions. This ensures that a user's request response time is not affected by the cache of other users, effectively preventing potential attacks.
- We conduct extensive experiments using different models and standard benchmarks to evaluate the partition mechanism's security and performance.

Results show that this approach improves security with minimal effect on first token latency, throughput and system startup time.

II. BACKGROUND

A. LLM Inference Optimization Techniques

Inference serving for LLMs involves optimizing model execution to meet latency and efficiency requirements, particularly under hardware constraints such as a limited GPU memory. To address these challenges, several techniques have been developed, including quantization and model distillation. Quantization [6] reduces the precision of model weights, resulting in lower memory requirements and faster computation. Model distillation [8] involves training a smaller model to replicate the behavior of a larger one, providing an efficient alternative without significantly compromising performance.

One of the most significant optimization techniques is KV cache prefix caching [4], [16], [19]. KV prefix caching stores tokens from previous user inputs along with their corresponding KV cache. If subsequent user prompts share the same prefix as the cached tokens, it reuses the KV cache for these prefix tokens, thereby reducing computational overhead and improving inference speed. This technique has been implemented in multiple inference frameworks, including vLLM [4], SGLang [19], and ChunkAttention [16], proving its effectiveness in various practical scenarios. In particular, the SGLang system introduces an efficient KV caching mechanism, RadixAttention, which uses a Radix tree to automatically manage shared prefixes in the KV cache. This structure supports prefix matching, insertion, and eviction, using an LRU strategy to manage cache entries.

In applications like Retrieval-Augmented Generation (RAG) [5] and long-context prompts, KV cache prefix caching can significantly improve inference performance by minimizing redundant computations.

B. Timing Side-Channel Attacks

Timing side-channel attacks [3], [9] infer sensitive information by observing execution time differences in a system, often applied to time-sensitive contexts like encryption and caching systems. Attackers measure subtle variations in operation time, memory access, or network latency to deduce hidden confidential data.

In LLM inference scenarios, there are currently a range of side-channel attack [12], [13], [17] methods based on inference optimization techniques. Specifically, in some techniques that employ shared cache optimization for inference [12], attackers can leverage differences in response time caused by shared cache mechanisms. As shown in Figure 1, ① when the victim inputs a prompt, the LLM inference backend stores the tokens of this prompt along with the corresponding KV cache in a node within a prefix tree, ② and after the victim, the attacker sends a prompt with the same prefix as the cached tokens.

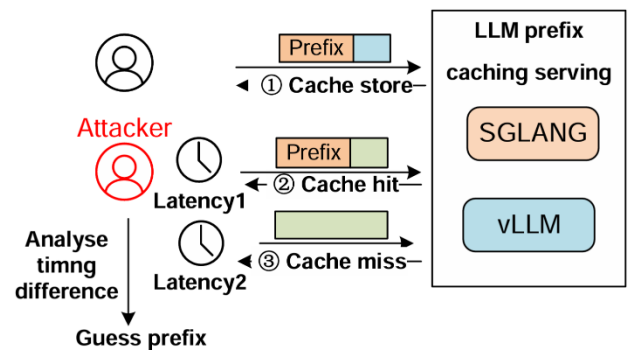


Figure 1. Timing side-channel leakage in prefix sharing LLM inference framework.

In this case, the prefix tree node hits, and the corresponding KV cache is reused for computation, resulting in a reduced response time for the attacker's prompt, which the attacker can observe, ③ then the attacker sends a prompt that does not share a prefix with the cached tokens, and the prefix tree node misses, requiring new KV cache computation and inference for the prompt, resulting in a longer response time compared to step , which the attacker can observe.

Therefore, in a multi-user shared inference architecture, the above attack method can be used to infer other users' prompts. Mitigating these vulnerabilities is crucial and practical for ensuring the privacy and security of LLM systems.

C. Threat Models

We assume the attacker and the victim are in different security domains, and the attacker cannot directly access the victim's secret data. Both the attacker and the victim interact with the same LLM inference framework, which relies on shared prefix caching for efficiency. The attacker can be a legitimate user who attempts to infer information about the victim's inputs by exploiting the shared caching mechanism.

The attacker performs a timing side-channel attack to glean secrets from the victim's domain. Specifically, the attacker actively sends crafted prompts to the LLM system, aiming to observe timing differences that indicate cache hits. By doing so, the attacker can infer whether certain prefixes were used by the victim, potentially revealing sensitive information.

Our threat model does not consider physical attacks, where the attacker gains direct access to the underlying hardware. Similar to existing defense mechanisms, we also do not address information leakage due to early termination times, which is inherently a program-level issue. A defense mechanism is considered secure if it prevents the attacker from distinguishing the memory or cache access patterns of the victim. In our context, the latency observed by the attacker should be independent of the victim's prefix cache status, ensuring that the attacker's timing measurements provide no useful information about the victim's activity.

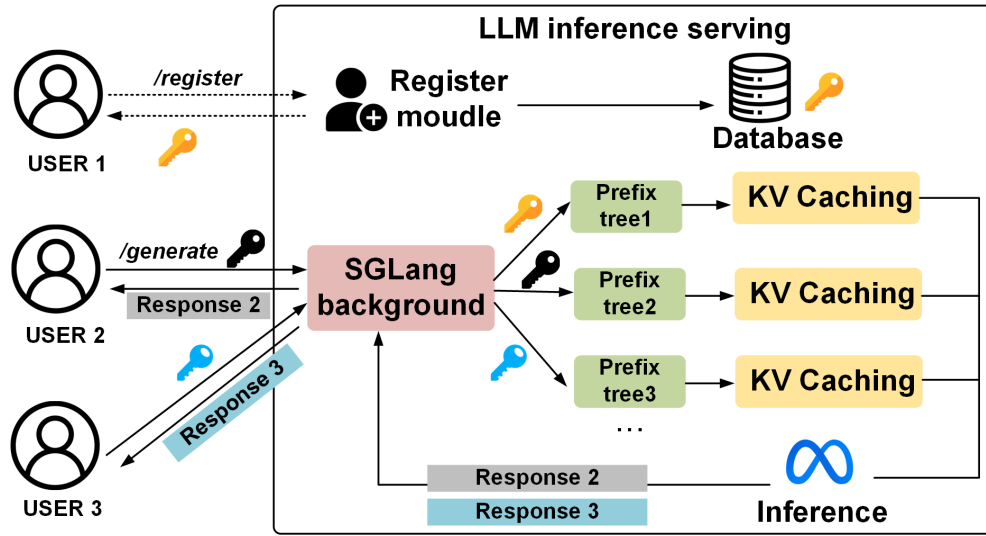


Figure 2. The architecture of user-isolation prefix sharing LLM inference framework

III. SYSTEM ARCHITECTURE

This section introduces a user-isolated KV cache partitioning mechanism based on SGLang inference framework to prevent timing side-channel attacks in multi-user LLM applications. As shown in Figure 2, the system has added a user registration and authentication module, linking the user frontend with the inference framework. The backend identifies the originating user of each request via the token included in the request. By establishing distinct prefix trees for matching based on user boundaries, this design implements KV cache partitioning [10], [11] across different users or applications. Consequently, each user has a dedicated cache space, and response times decrease only when the same user inputs identical prefixes, preventing timing side-channel vulnerabilities across different user sharing the same serving framework. A key design feature of this approach is that it maintains the mapping pool between requests and tokens, as well as between tokens and the corresponding KV cache, which maintains the performance of the system. The following section provides a detailed overview of the system's design and implementation.

A. User Registration and Authentication Module

To achieve user-level request isolation and secure cache management, the system implements a registration module to generate and manage a unique identifier (*token*) for each user, which is used for authentication by the inference backend.

- **Generation and storage of token:** upon receiving a registration request (*/register*), the inference frontend generates a unique token using the sha256 hashing algorithm to ensure that each user's identifier is unique and unpredictable. The token is stored in the database for subsequent user authentication.
- **Registration interface:** once registration is successful, the system returns the generated token to the user, which the user must securely retain for use in future requests. This step establishes each user's unique identity, forming the basis for the isolation mechanism.

- **Backend connection initialization and authentication:** To establish a connection to the backend, Users initialize the backend connection by passing their token (e.g., via `sgl.RuntimeEndpoint`) and setting it as the default backend for subsequent requests. Upon receiving the connection initialization request, the system verifies the token by querying the database. If the token is invalid or non-existent, the connection request is denied, thereby protecting backend resources from unauthorized access.

B. Request Transmission and Encapsulation

When users send inference requests, the token is embedded in the request header. Then, the backend extracts the token from the request header and encapsulates it along with the tokenized user input into a new data structure, which is then used for subsequent prefill and decode processes in the LLM. This encapsulated request object includes both the inference data and the unique user identifier, enabling isolated handling during prefix matching and cache management.

By connecting the frontend and backend, the system enables the backend to recognize user identity information. Encapsulating the user identity and input tokens into a single data structure allows the underlying KV cache's prefix caching to implement isolation based on user boundaries.

C. Cache Matching and Isolation via Prefix Tree

In order to realize user-bounded KV cache partitioning, we dynamically extend the single prefix tree used for matching in the backend of the inference framework to N . Whenever a new authenticated user initiates a request, the system automatically creates a new dedicated prefix tree in the form of a dictionary. When a returning user submits a request, it is directed to the corresponding prefix tree for matching. It should be noted that, based on the multi-round dialog scenario usually used in prefix caching, the likelihood of multiple non-attacker users sharing identical prefixes is low in a non-isolated system. As a result, each user is likely to create unique, non-overlapping branches

from the shared root node of the backend prefix tree. This design is thus theoretically reasonable and does not incur too much overhead. In this scenario, attackers are also prevented from inferring the prompt structure of other users based on the timing of cached responses to their own requests.

- **Independent prefix trees for users:** The system maintains independent prefix trees for each user within the prefix cache inference framework to ensure cache isolation at the prefix matching stage. The system creates a prefix tree for each user's *token* and stores it in a dictionary structure, *tree_cache*. In *tree_cache*, each key corresponds to a user's token, and each value is the associated prefix tree.
- **Prefix matching isolation:** Importantly, while the mapping pools between requests and tokens, and between tokens and the KV cache, remain shared, prefix matching is isolated by user. This allows the system to locate the prefix tree associated with the current user's request by referencing the *token* in *tree_cache*, ensuring that prefix matching only occurs within the user's designated cache.
- **Batch processing and efficiency:** When processing a batch of requests, the system identifies the appropriate prefix tree based on each request's token. The caching-requests method performs prefix matching exclusively within this user's prefix tree, returning matching prefix indices and node information to be updated within the request. This mechanism enhances inference efficiency by maintaining a shared mapping pool while preventing cross-user cache access during prefix matching.

By isolating prefix matching to requests with the same token while retaining the global mapping pool, the system achieves a balance between performance efficiency and secure cache isolation. This method defends against potential timing side-channel attacks while supporting efficient resource utilization in multi-user LLM environments.

IV. EVALUATION

In this section, we conduct security and performance evaluation of the cache partition prefix caching system on different models. In the security evaluation, we run two different users (attacker and victim) to verify whether, in the isolation system proposed in this paper, it is possible to determine if the prompt shares the same prefix with another user based on response time. If not, this indicates that the side channel has been eliminated. In performance evaluation, we compare the performance of the isolated system with that of the original system across different metrics, such as Time-To-First-Token (TTFT), throughput and system startup time, and are conducted in different scenarios, such as single-user and multi-user environment.

A. Experimental setup

The experiments were performed using SGLang version 0.2.6 [1], comparing the user-isolated cache partition

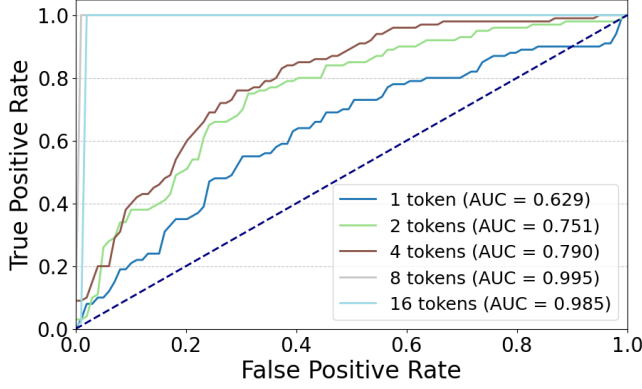
mechanism against the original unmodified version as the baseline. The setup details are as follows:

- **Model:** The experiments were performed using the *TheBloke/Llama-2-70B-Chat-GPTQ* and *meta-llama/Llama-2-13b* models.
- **Hardware:** All experiments were conducted using a 40GB NVIDIA A100 GPU, providing the necessary computational resources for LLM inference tasks.
- **Workload:** The following benchmark datasets were used to evaluate the system: GSM8K [14], which is used primarily for mathematical reasoning; MMLU [2], the Massive Multitask Language Understanding dataset that covers a wide range of disciplines including science, humanities, and social sciences; and Hellaswag [18], a commonsense reasoning dataset containing a large number of texts requiring commonsense judgment and reasoning. For ReAct agents [15], we extract the traces from the original papers.
- **Metrics:** We report three performance metrics: TTFT, throughput and system startup time. For TTFT, we execute a single program at a time without batching and report the average latency for multiple requests. For throughput, we simulate executing a task in batches using an LLM and calculate the number of requests that can be completed per second. For system startup time, we measure the time required to initialize the LLM inference system and make it ready for serving requests, comparing both isolated and non-isolated systems.

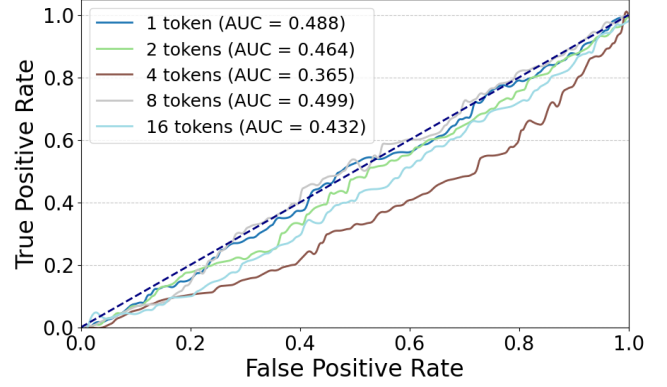
B. Security Evaluation

In our security evaluation, we tested SGLang [1] LLM inference system both with and without cache partition mechanisms. Specifically, we analyzed the ROC curves of response times observed when the attacker sent prompts with the same and different prefixes as the victim, which indicates the separability of the data. Under the partition isolation mechanism, a degraded ROC curve would suggest that exploiting timing side-channels to infer the target user's information is infeasible.

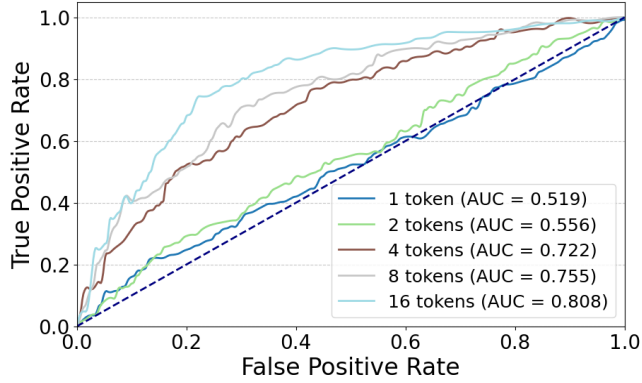
To evaluate the impact of a shared prefix from another user (victim) on the response time of the current user's (attacker) requests under both the initial system and the isolation system proposed in this paper, we first set an input prompt P for the victim. Additionally, we created six groups of prompts P_k for the attacker, each of the same length as P , where each group shares the first k tokens with P for $k = 0, 1, 2, 4, 8, 16$. Each group contains 1000 prompts. Next, after P was input by the victim in one program, the attacker, acting as a new user in another process, input six groups of prompts and recorded TTFT. In the evaluation, time samples from P_0 were labeled as 0, while samples from other groups were labeled as 1. Finally, ROC curves were plotted for P_k against P_0 for $k = 1, 2, 4, 8, 16$.



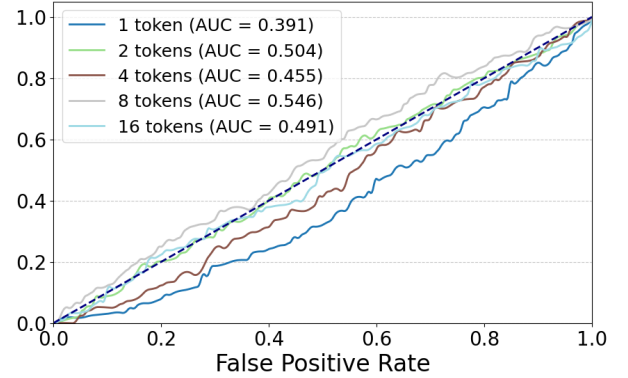
(a) ROC for non-isolated system on Llama-2-70B-Chat-GPTQ.



(b) ROC for user-isolated system on Llama-2-70B-Chat-GPTQ.



(c) ROC for non-isolated system on Llama-2-13b.



(d) ROC for user-isolated system on Llama-2-13b.

Figure 3. We tested whether cross-user prefix sharing impacts the response time of individual users in both the original and isolated systems on two different models, *Llama-2-13b* and *Llama-2-70B-Chat-GPTQ*. We plotted ROC curves for shared prefix lengths of 1, 2, 4, 8, and 16 to assess whether the new isolation system eliminates side-channel vulnerabilities. When the AUC drops below 0.5, it indicates that security has been achieved.

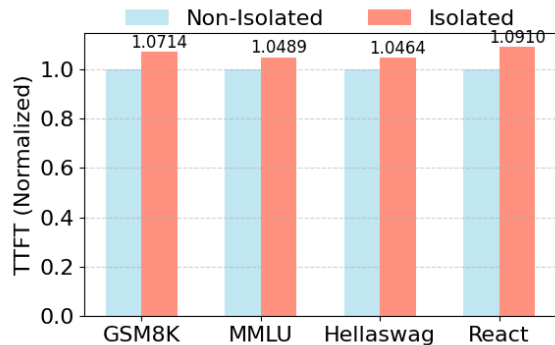
As shown in Figure 3, Figure 3(a) and 3(c) reveals that in a system without cache partitioning, cross-user shared prefixes reflect a significant caching effect on TTFT. As the length of the shared prefix increases, this effect becomes more pronounced. Even with only one shared token, the AUC value reaches above 0.62, and as the prefix length increases, it can reach close to 1. Additionally, in systems with larger model weights, this effect is even more evident and stable. Thus, it is evident that a severe side-channel vulnerability exists in the non-isolated system. In contrast, in the system with user isolation mechanisms proposed in this paper (as shown in Figure 3(b) and Figure 3(d)), the ROC curves become indistinguishable, the AUC drops below 0.5, and no correlation exists between response time and prefix length. This demonstrates that the cache partition mechanism eliminates side-channel vulnerabilities caused by cross-user shared LLM inference frameworks, preventing the attacker from using response times of their own requests to infer other users' input prompts, thus achieving security in a multi-tenant environment.

C. TTFT Evaluation on different scenarios

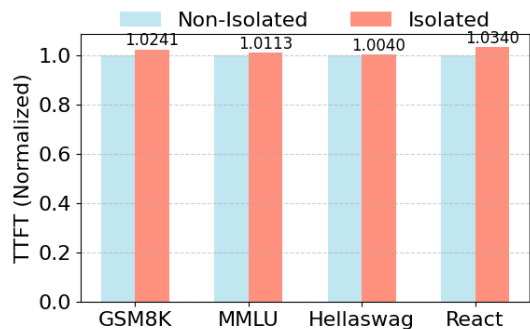
1) *Single-user scenarios*: To measure TTFT, we begin by selecting a few shot examples from different datasets to use as

prefixes, which are then combined with individual data entries and input into the SGLang system for inference. These examples are cached as prefix examples, with 5 shots examples used for GS8MK, 5 shots for MMLU, and 8 shots for Hellawag. For each user request, we set timestamps before and after execution and limit the output length to one token to accurately capture TTFT. Prior to sending inference requests, each user registers with the frontend of the inference framework to obtain a unique identification token, which is then included with each subsequent backend connection. The evaluation was conducted in both single-user and multi-user scenarios to assess how user isolation affects TTFT. In single-user scenario, we measured average TTFT for each dataset. For multi-user scenario, we simulated three concurrent users, each sending inference requests to the system, and recorded the average TTFT. Figure 4 shows TTFT for different datasets and models in single-user scenarios, which increased by an average of 6.44% and 1.8% under the cache partition isolation mechanism. Specifically, in Figure 4(a), on the larger model *Llama-2-70B-Chat-GPTQ*, TTFT increased by 7.14% on the GS8MK dataset, and by 4.89%, 4.64%, 9.10% on the MMLU, Hellaswag and React datasets, respectively. This increase in

latency is mainly attributed to the additional overhead involved in lookup and matching operations within the user-isolated prefix trees. Since the response time for a single request is generally within 0.1 seconds in inference frameworks that use cache optimization, the increased latency overhead is acceptable for regular users in practical scenarios. In the smaller model, due to the lower memory usage of the KV cache and faster request processing speed, there is essentially no significant overhead. The increase in latency is within acceptable limits, demonstrating that the additional security provided by user isolation comes at a relatively low performance cost.



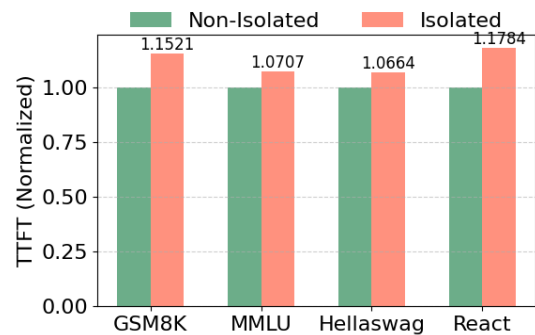
(a) On Llama-2-70B-Chat-GPTQ.



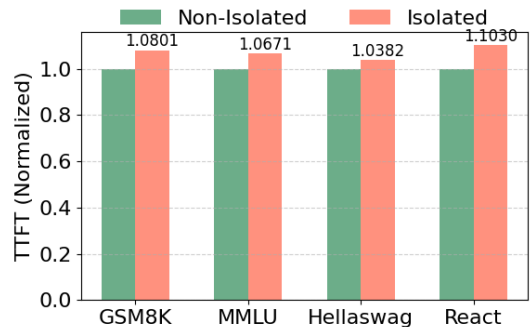
(b) On Llama-2-13b.

Figure 4. Normalized performance of TTFT between original systems(non-isolated) and the cache partition system(isolated) in single-user scenarios on different datasets and two different models Llama-2-13b and Llama-2-70B-Chat-GPTQ. Lower is better.

2) *Multi-users scenarios*: To further evaluate the impact of user isolation on TTFT performance in multi-user scenarios on Llama-2-70B-Chat-GPTQ model, we simulated three users simultaneously accessing the inference framework. Firstly, the users use the same prefix and the results are shown in the Figure 5(a), indicate that the TTFT for the isolated system compared to the non-isolated system increased by 15.21%, 7.07%, 6.64% and 17.84% for the GSM8K, MMLU, Hellaswag and React datasets. Compared with results from the single-user scenario, the performance of the user-isolated inference system shows a decline in the multi-user setting. This reduction primarily arises from the user isolation



(a) Different users have the same prefix.

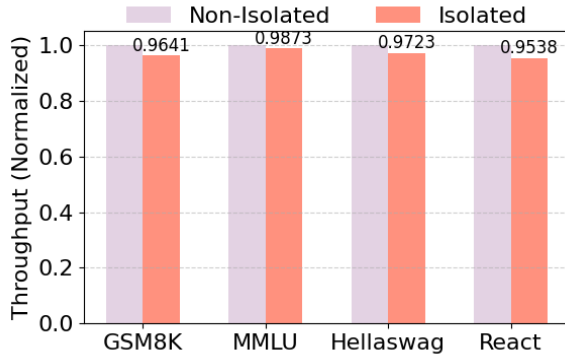


(b) Different users have different prefixes.

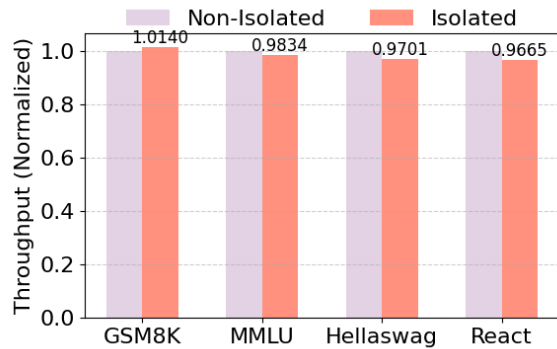
Figure 5. Normalized performance of TTFT between new isolated and original non-isolated systems in multi-user scenarios on different datasets. Figure 5(a) shows the TTFT comparison between the two systems when multiple users share the same prefix, shows the potential maximum overhead of the cache partition system. Figure 5(b) presents the TTFT comparison between the two systems when multiple users do not share prefixes, which is a more realistic and common scenario. Lower is Better.

mechanism, which prevents caching of the same prefix when sent by different users. Consequently, the system's KV cache requires more frequent scheduling, such as eviction, leading to a lower cache utilization rate. GS8MK and React shows a more performance drop due to their longer request length, averaging 1000 and 1500 tokens. This experiment is an extreme scenario, illustrating the potential maximum overhead of the cache partition system, where every user shares a single prefix, requiring the isolation system to generate a separate prefix tree for each user to support cache partitioning. However, in real-world applications, cross-user interactions rarely involve such extensive shared prefixes, as prefix caching optimization is primarily intended to support single-user multi-turn conversation history and document reading scenarios. Therefore, in practice, even with shared GPU memory across users, there is a very low probability that the KV cache stored for multiple users would overlap, unless it is for an attack purpose. Therefore, we conducted further experiments to evaluate the TTFT performance overhead of the cache partition system in a multi-user scenario where user prefixes are not shared. The results are shown in Figure 5(b). The TTFT for the isolated system compared to the non-isolated system increased by 8.01%, 6.71%, 3.82% and 10.3% relatively, essentially keeping the performance consistent with

a single-user environment shown in Figure 4(a). Based on the experimental results, we can conclude that the overhead of the user-isolated prefix caching system remains within an acceptable range in realistic scenarios.



(a) On Llama-2-70B-Chat-GPTQ.



(b) On Llama-2-13b.

Figure 6. Normalized performance of the throughput of the original system (non-isolated) and the partitioned isolation system (isolated) on different datasets and two different models Llama-2-13b and Llama-2-70B-Chat-GPTQ. Higher is better.

D. Throughput Evaluation

In the throughput evaluation, we input batched task requests from different datasets into the inference framework in parallel for both the cache partitioning system and the original system, and calculated the average number of requests executed per second. We used a batch of 64 requests, testing an average of 1000 data on every dataset. The experimental results are shown in the Figure 6.

The experimental results show that on different models, when simulating users executing large numbers of requests in parallel by batch, the cache partition system proposed in this paper demonstrates excellent throughput, and the maximum throughput overhead is only 4.62%. The overhead may be caused by the identity authentication and querying the user's matching prefix tree. But as the system does not modify the mechanisms for scheduling and eviction of caches generated during computation in the inference framework, the mapping relationship between tokens in the prefix tree and their corresponding KV cache are effectively preserved. This reduces time overhead caused by frequent scheduling. The

experimental results confirm the feasibility of this partitioning approach in high-concurrency systems.

E. System Startup Time Evaluation

System startup time refers to the time required to initialize the LLM inference system and make it ready for serving requests. To measure the system startup time, we calculated the time interval from the execution of the inference framework startup script in the terminal to the point when the framework completes initialization and is ready to receive user input. The evaluation result is presented in Table 1. The difference in startup time is minimal, indicating that the overhead introduced by user isolation is negligible.

TABLE I. STARTUP TIME COMPARISON

System	Startup Time (s)
Non-Isolated	17.267
Isolated	17.743

The slight increase in startup time for the isolated system is attributed to the additional overhead involved in creating and connecting to the database used for managing user tokens. Experimental results indicate that the impact of user registration and the isolation mechanism on system initialization efficiency is negligible.

V. DISCUSSION

A. Security and Performance Trade-Off

The proposed user-boundary prefix cache isolation mechanism introduces a trade-off between enhanced security and increased latency. Experimental results showed that while TTFT increased by around 6.44% and the throughput decreased by 3.06%, this additional latency is generally within acceptable limits for most real-world applications. The slight performance degradation is outweighed by the significant improvement in mitigating timing side-channel attacks, ensuring that sensitive user information remains protected in multi-user environments.

B. Future Optimization Directions

Future work could focus on further optimizing the isolation mechanism to reduce the slight overhead introduced by the current implementation. One potential area for improvement is implementing a detection mechanism to identify potential attack prompts. This mechanism would be maintained on the backend and evaluate each user request to determine whether it could be classified as a potential attack.

For instance, if the number of prefix cache hits is consistently less than the request length by a small margin, it could indicate that the user is attempting to guess cached prefixes by manipulating tokens. Another approach could involve designing a sliding window to monitor the number of prefix hits over time—if the number of hits within the window is incrementally increasing, it may suggest suspicious behavior. By incorporating these detection mechanisms, the system can proactively identify and mitigate potential timing

side-channel attacks, further enhancing the security of multi-user environments while maintaining efficient performance.

VI. CONCLUSION

In this paper, we propose a user-boundary-based LLM inference cache partitioning mechanism. By partitioning users' caches, this mechanism prevents attackers from inferring other users' private information based on their own response times. Experimental results demonstrate that this partitioning system achieves very high levels of security and performance. This mechanism can be widely applied in service frameworks that optimize LLM inference efficiency through shared caches.

REFERENCES

- [1] Sglang is yet another fast serving framework for large language models and vision language models. <https://github.com/sgl-project/sglang>, 2024.
- [2] Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. Measuring massive multi-task language understanding. *arXiv preprint arXiv:2009.03300*, 2020.
- [3] Peter Horvath, Lukasz Chmielewski, Leo Weissbart, Lejla Batina, and Yuval Yarom. Barracuda: Bringing electromagnetic side channel into play to steal the weights of neural networks from nvidia gpus. *arXiv preprint arXiv:2312.07783*, 2023.
- [4] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th Symposium on Operating Systems Principles*, pages 611–626, 2023.
- [5] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in Neural Information Processing Systems*, 33:9459–9474, 2020.
- [6] Ji Lin, Jiaming Tang, Haotian Tang, Shang Yang, Wei-Ming Chen, Wei-Chen Wang, Guangxuan Xiao, Xingyu Dang, Chuang Gan, and Song Han. Awq: Activation-aware weight quantization for on-device llm compression and acceleration. *Proceedings of Machine Learning and Systems*, 6:87–100, 2024.
- [7] Zichang Liu, Aditya Desai, Fangshuo Liao, Weitao Wang, Victor Xie, Zhaozhuo Xu, Anastasios Kyrillidis, and Anshumali Shrivastava. Scissorhands: Exploiting the persistence of importance hypothesis for llm kv cache compression at test time. *Advances in Neural Information Processing Systems*, 36, 2024.
- [8] Chenlin Meng, Robin Rombach, Ruiqi Gao, Diederik Kingma, Stefano Ermon, Jonathan Ho, and Tim Salimans. On distillation of guided diffusion models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 14297–14306, 2023.
- [9] Hoda Naghibijouybari, Ajaya Neupane, Zhiyun Qian, and Nael Abu-Ghazaleh. Rendered insecure: Gpu side channel attacks are practical. In *Proceedings of the 2018 ACM SIGSAC conference on computer and communications security*, pages 2139–2153, 2018.
- [10] Dan Page. Partitioned cache architecture as a side-channel defence mechanism. *LACR Cryptol. ePrint Arch.*, page 280, 2005.
- [11] Kartik Ramkrishnan, Stephen McCamant, Pen Chung Yew, and Antonia Zhai. First time miss: Low overhead mitigation for shared memory cache side channels. In *Proceedings of the 49th International Conference on Parallel Processing*, pages 1–11, 2020.
- [12] Linke Song, Zixuan Pang, Wenhao Wang, Zihao Wang, XiaoFeng Wang, Hongbo Chen, Wei Song, Yier Jin, Dan Meng, and Rui Hou. The early bird catches the leak: Unveiling timing side channels in llm serving systems. *arXiv preprint arXiv:2409.20002*, 2024.
- [13] Jiankun Wei, Abdulrahman Abdulrazzag, Tianchen Zhang, Adel Muurseppe, and Gururaj Saileshwar. Privacy risks of speculative decoding in large language models. *arXiv preprint arXiv:2411.01076*, 2024.
- [14] Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Tom Griffiths, Yuan Cao, and Karthik Narasimhan. Tree of thoughts: Deliberate problem solving with large language models. *Advances in Neural Information Processing Systems*, 36, 2024.
- [15] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. *arXiv preprint arXiv:2210.03629*, 2022.
- [16] Lu Ye, Ze Tao, Yong Huang, and Yang Li. Chunkattention: Efficient self-attention with prefix-aware kv cache and two-phase partition, 2024.
- [17] Itay Yona, Ilia Shumailov, Jamie Hayes, and Nicholas Carlini. Stealing user prompts from mixture of experts. *arXiv preprint arXiv:2410.22884*, 2024.
- [18] Rowan Zellers, Ari Holtzman, Yonatan Bisk, Ali Farhadi, and Yejin Choi. Hellaswag: Can a machine really finish your sentence? *arXiv preprint arXiv:1905.07830*, 2019.
- [19] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Jeff Huang, Chuyue Sun, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E Gonzalez, et al. Efficiently programming large language models using sglang. *arXiv e-prints*, pages arXiv–2312, 2023.